

大厂 AI Coding 笔试技巧汇总

适用场景：允许或要求使用平台 AI 的仓库修改题、工程项目题、Code Repository 题和限时协作编程题。

核心目标：在有限时间内，把模糊需求转成可验收任务，约束 AI 小步实现，并用测试和解释证明结果。

小红书经验原图

秋招 AI Coding 笔试怎么做？

今年秋招最大的变化 —— 笔试不再防 AI，而是直接考你用 AI 的能力。

蚂蚁、京东、阿里等大厂已经把 AI Coding 作为正式笔试环节，牛客也上线了 AI 协作能力测评。这篇帖子把我的备考方法和实战经验整理出来

📌 先搞清楚：AI Coding 笔试考什么？

和传统算法笔试完全不一样。它不考你手写代码的能力，考的是：

- 你能不能看懂一个复杂的业务需求（README 文档）
- 你能不能把任务拆解成 AI 能执行的步骤
- 你和 AI 的交互过程是否高效、有逻辑
- 最终交付的代码能不能跑通测试用例

三个评分维度：**Prompt 设计能力 + 工具调用能力 + 任务拆解与业务交付能力**

通俗说：你是 "AI 项目经理"，AI 是你的 "开发实习生"，你指挥得好不好决定分数。

🔥 核心方法论：6 步流程

Step 1 | 读题分析，禁止直接写代码

拿到题目后，第一轮对话让 AI 做的事：

- 分析题目涉及哪些核心对象、状态、接口
- 梳理实现顺序和依赖关系
- 列出可能的边界条件和异常情况
- 输出一份 "做题地图"

这步相当于画施工图纸。没有图纸就动工 = 地基不稳后面全崩

Step 2 | 分模块开发，写完即测

不要让 AI 一次性生成全部代码！按优先级分轮实现：

- 第一轮：基础骨架（服务启动、初始化、时钟）
- 第二轮：主流程（正常业务路径走通）
- 第三轮：异常 / 边界 / 幂等处理

每写完一个模块就去终端跑测试。能过就锁定，不过就立刻修。

AI Coding 考察重点与步骤 1-2

小红书号: 27491519884

💡 本质认知：这不是代码能力考试

AI Coding 考的是你的**工程管理能力**—— 能不能把一个模糊的需求拆成可执行的任务、能不能控制 AI 不乱来、能不能做质量把关。

这与实际工作中用 AI 写代码是一回事：你不需要每行代码都自己写，但你得知道什么是对的、怎么验证、出了问题怎么定位。

某种意义上，这比纯算法笔试更接近真实工作场景。练好了不只是为了笔试，入职后天天用得上。

© 2024 小红书

AI Coding 的工程管理本质

⚠️ 高频踩坑点

坑	为什么
首轮就让 AI 全量实现	基础字段和状态没稳就往上堆，后面修不动
只跑平台样例就交卷	样例只测 happy path，隐藏用例全是边界
不限制 AI 的修改范围	AI 自由发挥 = 一改全崩
忽略幂等	重复请求必须幂等处理，不处理就扣分
用了真实时间	题目有虚拟时钟就必须用虚拟时钟，踩一次全盘皆输
账务只看余额	必须维护完整流水（发放 / 占用 / 使用 / 退回 / 到期）
得分卡住死磕同一个会话	新开会话让 AI 换个思路，经常有奇效

🔧 实用技巧

1. **语言选 Python**——AI 生成 Python 代码的准确率最高，Java/Go 跑分容易低
2. **预留充足时间**—— 别只留 20 分钟，30-40 分钟比较从容
3. **每轮加护栏 prompt**—— 结尾附上："不重构其他模块、不删已有接口、非法请求无副作用、只用虚拟时钟"
4. **得分下降立刻回退**—— 有时候改着改着分反而降了，果断回退到高分版本
5. **卡住就新开会话**—— 同一个上下文聊太多轮 AI 会糊涂，新窗口重新来效果更好

📊 得分区间参考

- **40 分以下**：大概率是首轮全量实现 + 没自测
- **40-55 分**：主流程跑通了但边界没覆盖
- **55-70 分**：有自测用例 + 分模块开发
- **70 分以上**：全覆盖 + 流水完整 + 幂等到位（大佬水平）

大部分人用上面的流程稳定在 60+，已经是不错的成绩了。

小红书

小红书号 27491518884

AI Coding 高频踩坑、实用技巧与经验分数区间

Step 3 | 报错精准喂给 AI

跑测试挂了？把完整的错误日志贴给 AI，并且限制它：

- "只修这一个 bug"
- "不要重构其他已通过模块"
- "保持其他接口行为不变"

不加限制的后果：AI 一兴奋给你重写半个项目，之前过的测试全挂了

Step 4 | 自己造测试用例

别等平台的隐藏用例来挂你。主动让 AI 生成刁钻测试：

- 正常路径
- 非法请求（参数错误、越权）
- 幂等测试（重复操作）
- 时间边界（到期、跨周期）
- 多批次 / 并发场景
- 终态验证

真实经验：有人靠这一步从 42 分直接跳到 64 分。隐藏用例测的就是这些。

Step 5 | 对照 README 逐条验收

快结束前，让 AI 对照原始需求文档逐条 check：

- 每个接口都实现了吗？
- 返回格式对不对？
- 文档要求的特殊规则覆盖了吗？
-

Step 6 | 交卷前让 AI 自审

最后 5 分钟，给 AI 一个 prompt：

"你现在是代码评审官。请检查以下维度：服务启动是否正常、ID 自增逻辑、字段状态流转、幂等处理、时钟使用（是否误用真实时间）、账务流水完整性。列出所有疑点。"

发现问题能改就改，改不了也能心里有数。

AI Coding 步骤 3-6：调试、测试、验收与自审

1. 先建立正确认知

AI Coding 笔试通常不是单纯考“手写代码速度”，也不是让 AI 无约束地替你完成项目。更接近真实开发工作的考察链路是：

理解需求 -> 定位代码 -> 拆解任务 -> 指挥 AI -> 审查改动 -> 测试验证 -> 解释取舍

结合公开平台说明，可以把考察重点归纳为四类：

1. **需求理解**：能否从 README、题面和现有代码中找出验收条件。
2. **任务拆解**：能否把复杂目标拆成 AI 可以可靠执行的小任务。
3. **AI 协作**：能否提供充分上下文、限制修改范围并纠正错误建议。
4. **工程交付**：能否通过构建、测试、边界检查和代码讲解证明结果。

CodeSignal 会向招聘方提供候选人与 AI 的完整对话和会话回放；HackerRank 的相关模式可记录 AI 对话、IDE 活动和 Diff，并评估候选人如何理解问题、采纳建议和推进实现。因此，“多问 AI”不等于高分，关键是每次交互是否有效、是否体现判断力。

2. 信息来源与事实边界

2.1 平台公开信息可以确认的部分

- 牛客公开的 AI Coding 挑战采用真实工程任务，强调借助平台 AI 完成需求理解、方案生成、代码修改、调试和可运行结果提交。
- CodeSignal 提供 Full AI Co-Pilot 和 Guided Support 两种协作方式，并记录候选人与 AI 的交互过程。
- HackerRank 区分 Guarded 与 Unguarded 模式；具体可用能力可能包括 Ask、Plan、Agent、内联补全和 Diff View。
- 某场考试允许使用 AI，不代表允许使用任意外部 AI。应以邀请邮件和考试页面的具体规则为准。

2.2 小红书图片中的经验性结论

图片提供的“六步流程”、小范围修改、主动补边界测试、逐条对照 README 等方法具有较强可迁移性，已整合进本文。

以下内容只能视为帖子作者的个人经验，不能当作统一规则：

- “Python 一定比 Java/Go 更容易得高分”；
- 固定的 40/55/70 分数区间；
- 某位考生通过补测试从 42 分提升到 64 分；
- 蚂蚁、京东、阿里等公司的具体覆盖范围；
- “卡住时新开 AI 会话一定有效”。

语言、评分标准、模型能力和题型都由具体平台与企业配置决定。应优先选择题目 Starter Repo 的原语言、平台完整支持的语言，以及自己最能调试和解释的语言。

3. 开考后的六步执行流程

Step 1: 读规则、跑基线、画“做题地图”

先确认：

- 只能使用平台 AI，还是允许外部工具；
- AI 是 Guarded 还是可以直接修改文件；
- 是否允许联网、查文档、复制粘贴；
- 是否录屏、监考并保存 AI 对话；
- 提交方式、时间限制和评分方式。

然后运行现有的启动、构建和测试命令，记录原始状态。不要修改代码后才第一次跑测试，否则无法判断失败是原项目已有，还是本次改动引入。

第一轮 AI 对话只做分析，不写代码：

请先不要修改代码。阅读题面、README 和当前仓库后：

1. 提取所有可验收的 PO 要求；
2. 列出核心对象、状态、接口和依赖关系；
3. 定位可能需要修改的文件和调用链；
4. 列出边界条件、异常情况和隐藏测试高风险点；
5. 给出最小实现顺序，并标出仍不确定的地方。

把结果压缩成一张验收表：

类别	要回答的问题
输入	参数格式、空值、非法值和权限要求是什么？
状态	有哪些状态，允许如何流转？
接口	方法签名、返回格式和错误行为是否固定？
依赖	数据库、缓存、时钟、第三方服务是否可替换或模拟？
副作用	写库、发消息、扣款等操作是否允许重复执行？
验收	哪些公开测试、构建命令和文档要求必须满足？

Step 2：分模块、小步实现、每步验证

不要让 AI 首轮生成整个项目。优先按垂直闭环推进：

1. **最小骨架**：项目可以启动，接口和数据结构就位。
2. **主流程**：先让正常业务路径完整跑通。
3. **异常与边界**：补非法请求、重复操作、超时、并发和终态限制。

每一轮给 AI 明确任务合同：

只完成 [具体功能]，修改范围限定为 [文件/类/函数]。
 必须保持 [接口/已有行为] 不变。
 不要新增依赖，不要重构其他模块，不要删除已有接口。
 完成后列出 Diff、关键假设和应运行的测试。

每完成一个模块就运行相关测试。通过后再进入下一步，避免多个未经验证的改动叠在一起。

Step 3: 精准报错, 限制修复范围

测试失败时, 把完整错误信息、复现步骤和相关代码上下文交给 AI, 但不要只说“还是不行”。

当前失败命令: [命令]
完整错误信息: [错误日志]
预期行为: [预期]
实际行为: [实际]

先不要重写实现:

1. 给出最可能的三个根因;
2. 为每个根因提供最小验证步骤;
3. 只修复确认的这个 Bug;
4. 不重构其他已通过模块, 保持公共接口行为不变。

如果 AI 连续两轮没有推进, 先检查上下文是否混乱、错误日志是否完整、任务边界是否过大。重新开会话可以作为最后手段, 但新会话必须重新提供题面、当前状态、已通过测试和剩余问题, 不能指望模型自动继承上下文。

Step 4: 主动设计隐藏用例

不要只满足平台示例或 happy path。根据题型主动覆盖:

- 正常路径;
- 空值、越界、格式错误和不存在的数;
- 未授权、越权和非法状态流转;
- 重复请求与幂等性;
- 超时、重试、跨周期和虚拟时钟;
- 多批次、并发和执行顺序;
- 失败后的回滚、补偿和最终一致性;
- 服务重启、资源释放和终态操作。

图片提到的“虚拟时钟、幂等和账务流水”是很好的业务题检查项, 但不是每道题都需要。应从 README、接口和领域状态推导边界, 不要机械套用。

可使用以下测试提示词:

请根据 README 和当前实现设计刁钻测试, 优先寻找隐藏测试风险。
覆盖正常路径、非法参数、重复请求、状态边界、时间边界、并发和终态。
先列测试矩阵及预期结果, 不要立即修改业务代码。

Step 5: 逐条对照 README 验收并审查 Diff

对照原始需求逐项检查:

- 每个要求是否都有对应实现或测试证据;
- 接口签名、状态码和返回格式是否完全一致;
- 是否误改无关文件或公共接口;
- 是否新增不允许的依赖、硬编码或全局状态;
- 是否吞掉异常、关闭校验或通过删除测试“解决”问题;

- 原有测试和新增测试是否全部通过；
- 构建、启动和提交命令是否可重复执行。

AI 的总结不是证据。最终以实际 Diff、构建结果、测试结果和运行行为为准。

Step 6: 交卷前自审，并准备独立讲解

最后一轮可以让 AI 作为代码审查者：

你现在是代码评审者。请只审查，不要直接重构。
根据 README、当前 Diff 和测试结果检查：

1. 是否遗漏验收要求；
2. 接口与状态流转是否正确；
3. 是否存在空值、越权、幂等、时间、并发和资源释放问题；
4. 是否有真实时间、硬编码、隐藏副作用或无关修改；
5. 哪些问题必须交卷前修，哪些可以记录为后续优化。

请按严重程度列出证据位置。

交卷前确保自己能脱离 AI 回答：

- 程序入口和主要调用链是什么？
- 为什么选择当前方案？
- AI 哪些建议被接受、修改或拒绝？
- 哪些边界条件已经验证？
- 仍有哪些已知限制？
- 如果再给半小时，下一步会做什么？

4. 时间分配模板

4.1 60 分钟题

时间	任务	阶段产物
0-5 分钟	读规则、确认权限、浏览题目	工具边界与提交要求
5-12 分钟	跑基线、定位入口、生成做题地图	验收表与修改范围
12-35 分钟	完成 P0 主流程	可运行的最小闭环
35-48 分钟	补异常与高风险边界	自造测试与修复结果
48-55 分钟	全量测试、构建、审查 Diff	可复现验证证据
55-60 分钟	清理调试代码、最终自审、提交	最终版本与讲解提纲

4.2 90 分钟题

时间	任务
0-15 分钟	规则、基线、调用链和任务拆解
15-50 分钟	骨架与主流程
50-70 分钟	边界、异常和隐藏测试风险
70-82 分钟	全量测试、构建与 Diff Review
82-90 分钟	回归高分版本、清理并提交

时间表不是死规定。原则是至少保留最后 15%-20% 的时间用于全量验证和提交，不要直到倒计时结束仍在大规模生成代码。

5. 高频踩坑与修正动作

踩坑	后果	修正动作
首轮让 AI 全量实现	需求误解会扩散到全项目	先做题地图，再分模块实现
只跑平台样例	示例通常只覆盖正常路径	主动补非法、重复、时间和状态边界
不限制修改范围	AI 容易重构已通过模块	每轮注明文件、接口和禁止事项
修改后分数下降仍继续叠加	丢失可用版本	保留通过节点，回退后重新缩小任务
忽略幂等	重复请求产生重复副作用	设计请求标识、唯一约束或状态检查
直接使用真实时间	时间相关测试不稳定	注入时钟或使用题目提供的虚拟时间
账务题只看余额	无法验证完整业务过程	校验发放、占用、使用、退回、到期流水
AI 连续多轮空转	上下文被错误假设污染	补全证据、缩小问题，必要时新会话
为了“架构漂亮”大重构	引入大量回归风险	以最小合格改动为目标
能运行但无法解释	暴露对 AI 的过度依赖	交卷前关闭 AI，独立走读代码

6. 语言与工具选择原则

按以下优先级选择语言：

1. Starter Repo 和现有测试使用的语言；
2. 平台依赖、SDK 和运行环境支持最完整的语言；
3. 自己最熟悉、最能快速定位错误和解释实现的语言；
4. AI 在该题技术栈上的生成稳定性。

Python 通常代码较短、AI 示例丰富，适合原型和数据处理；Java 在明确领域模型、类型约束和 Spring 项目中可能更稳；Go 在并发和服务题中也有优势。不要为了帖子中的经验结论，临场更换自己不熟悉的语言。

7. 针对 Agent/AI 应用开发岗位的训练重点

优先练习以下仓库级任务：

- FastAPI/Spring Boot 现有接口 Bug 修复；
- LLM API 的超时、重试、限流和结构化输出；
- Agent 工具参数校验、权限边界和失败恢复；
- RAG 的召回、重排、空结果和引用链路；
- 缓存一致性、数据库事务、幂等和并发；
- 日志、测试和可观测性补全；
- 对现有项目做最小功能增量，而不是从零生成 Demo。

每次训练都保留四类证据：题面与验收表、AI 对话摘要、最终 Diff、测试与构建结果。

8. 七天训练安排

天数	训练主题	验收标准
Day 1	只做题目拆解与调用链定位	10 分钟内生成准确做题地图
Day 2	小范围 Bug 修复	仅改必要文件并通过回归测试
Day 3	新增一个后端接口	主流程、非法输入和权限测试通过
Day 4	幂等、时间与状态机	重复请求和时间边界行为正确
Day 5	LLM/Agent 工具题	处理超时、失败、结构化输出和审计
Day 6	60 分钟完整模拟	留出最后 10 分钟做全量验证
Day 7	复盘与二次模拟	能独立解释方案、AI 错误和取舍

9. 交卷前一分钟清单

- [] 已确认 AI 与外部工具使用规则。
- [] 已保留并理解题目要求和提交方式。
- [] 主流程、异常路径和关键边界均有验证。
- [] 全量测试与构建已实际运行。
- [] 没有调试输出、硬编码凭据或无关文件。
- [] 没有删除测试、关闭校验或吞掉异常。
- [] Diff 中每一处改动都能解释原因。
- [] 当前版本优于最后一个已知通过版本。

- [] 提交物、文件路径和入口符合题目要求。
- [] 即使关闭 AI，也能讲清实现、风险与取舍。

10. 参考资料

- 牛客 AI Coding 笔试挑战杯
 - CodeSignal: AI-Assisted Coding Assessments and Interviews
 - HackerRank: AI-Assisted Interviews
 - HackerRank: AI Fluency Evaluation
 - 真实公司面试 AI Coding 项目题与通用方法
-

****一句话记忆：****先把题目变成验收清单，再让 AI 小步改；每步看 Diff、跑测试，最后由自己对结果负责。